

HƯỚNG DẪN QUY TRÌNH PHÁT TRIỂN & HỢP TÁC SPRINT 2

Tài liệu này hướng dẫn chi tiết về **quy trình hợp tác phát triển mới đã được rút gọn** của dự án. Tất cả các thành viên trong đội ngũ phát triển (Backend .NET, Backend Spring, Frontend) cần đọc kỹ và tuân thủ các quy tắc dưới đây trong suốt Sprint 2.

1. BỐI CẢNH & THAY ĐỔI CỐT LÕI

Để đẩy nhanh tiến độ và loại bỏ các thủ tục rườm rà, chúng ta đã **tối giản hóa quy trình Git và quản lý Issue**:

- 1. Gộp Issue:** Đóng toàn bộ các task nhỏ nhất. Thay vào đó, mỗi mảng tính năng lớn chỉ có duy nhất **1 issue lớn** đại diện trên GitHub.
 - 2. Tự do đặt tên & Commit:** Không còn bắt buộc tiền tố tên nhánh (`feature/` , `fix/` ...) hay quy chuẩn tin nhắn commit nghiêm ngặt (Conventional Commits).
 - 3. Bảo vệ nhánh cốt lõi:** Nhánh `dev` và `main` được bảo vệ bằng Git Hooks, ngăn chặn tuyệt đối việc đẩy code trực tiếp (`git push`).
-

2. QUY TRÌNH GIT & PHÂN NHÁNH MỚI

Tất cả lập trình viên bắt buộc phải tuân theo luồng làm việc 3 bước sau:

Bước 1: Tạo nhánh làm việc riêng từ `dev`

Trước khi code, hãy cập nhật code mới nhất và tạo một nhánh tuân thủ đúng quy chuẩn sau:

- Quy chuẩn đặt tên nhánh:** `<github-username>/<issue-id>-<short-name>`
 - Trong đó:
 - `<github-username>` : Tên tài khoản GitHub của bạn (ví dụ: `Ahug05` , `ToTrieuTien` , `fong-gif`).
 - `<issue-id>` : Số thứ tự Issue trên GitHub (ví dụ: `71` , `72` , `73`).
-

- **<short-name>** : Tên ngắn gọn mô tả tính năng bằng tiếng Anh, viết thường, phân tách bằng dấu gạch ngang (ví dụ: `users-crud` , `manage-floors`).

Ví dụ thực tế: Để bắt đầu làm CRUD Users (Issue #71, do dev Ahug05 phụ trách), bạn chạy các lệnh sau:

✦ Bước 2: Lập trình, Commit và Push

Bạn có thể commit bất kỳ lúc nào với mọi loại tin nhắn commit (thích hợp cho commit nháp khi chuyển nhánh). Đẩy nhánh của bạn lên remote:

Ví dụ thực tế:

✦ Bước 3: Tạo Pull Request (PR) để hợp nhất vào `dev`

Lên giao diện GitHub, tạo một **Pull Request** hướng về nhánh `dev` để kiểm tra và hợp nhất code.

Ví dụ thực tế: Tạo Pull Request từ nhánh `Ahug05/71-users-crud` cần merge vào nhánh `dev` .

Lưu ý

⚠ **Chú ý:** Lệnh `git push origin dev` sẽ bị chặn trực tiếp bởi Git Hook để bảo vệ mã nguồn chung. Bạn chỉ có thể đưa code vào `dev` bằng Pull Request.

💡 3. NGUYÊN TẮC "PULL REQUEST NHỎ - MERGE SỚM"

Mặc dù cả Sprint 2 bạn chỉ có 1 Issue lớn, nhưng **đừng đợi đến cuối Sprint mới tạo PR**. Việc ôm code quá lâu sẽ dẫn tới xung đột (conflict) nghiêm trọng khi ghép code chung.

- **Cách làm đúng:** Khi bạn hoàn thành xong một module nhỏ độc lập (ví dụ: xong Entity và Repository của một bảng), hãy tạo ngay một PR nhỏ để merge vào `dev` .
- Điều này giúp code của cả đội luôn được tích hợp liên tục và giảm thiểu tối đa rủi ro lỗi xung đột.

📁 4. PHÂN CHIA ISSUE SPRINT 2 TRÊN GITHUB

Toàn bộ công việc của Sprint 2 hiện tại được gom lại thành **4 Issue lớn** sau:

Issue ID	Tên Issue	Người phụ trách	Mô tả tóm tắt
#71	Quản trị dữ liệu nền cốt lõi	Ahug05	CRUD User, Card, Vehicle Type, Pricing Rule trên .NET Core. Cấu hình JWT Token Claims.
#72	Quản trị cấu trúc bãi đỗ xe	ToTrieuTien	CRUD Floor, Area, Slot, Gate trên .NET Core. Ràng buộc nghiệp vụ trùng lặp cấu trúc bãi.
#73	APIs đọc dữ liệu & Khung Dashboard	fong-gif	Viết APIs chỉ đọc public (info, slots bận/trống, pricing) & Dashboard trên Spring Boot.
#74	UI Đăng nhập & Quản lý dữ liệu nền	*Frontend Dev*	Giao diện Login, Route Guard bảo vệ, CRUD các trang quản trị nền bãi xe trên React.

5. HƯỚNG DẪN KỸ THUẬT & CẤU HÌNH CỤC BỘ

Không lưu thông tin đăng nhập trong code (Cấu hình Biến môi trường)

Để bảo mật dự án và tránh rò rỉ thông tin đăng nhập thực tế của Supabase lên GitHub, toàn bộ chuỗi kết nối Database và JWT Secret đã được chuyển đổi sang dạng **đọc qua biến môi trường**.

Hướng dẫn tạo và chạy file cấu hình môi trường cục bộ (env):

Cách 1: Sử dụng PowerShell (Khuyến dùng trên Windows)

1. Tạo một file tên là `env.ps1` ở thư mục gốc của dự án (thư mục chứa file `.gitignore`).
2. Viết nội dung cấu hình kết nối database của bạn vào file đó:

```
`powershell
```

Kết nối DB cho .NET & Spring Boot

```
$env:DB_URL="jdbc:postgresql://localhost:5432/parking_db" # Hoặc URL kết nối database Supabase của bạn
```

```
$env:DB_USERNAME="postgres"
```

```
$env:DB_PASSWORD="your_password"
```

Khóa bí mật JWT

```
$env:JWT_SECRET="DEVELOPMENT_SECRET_KEY_FOR_LOCAL_TESTING_ONLY_2026_SWP391"
```

```
`
```

3. Mỗi khi mở cửa sổ PowerShell mới để chạy code hoặc test, hãy chạy file này trước (gọi là Dot-Sourcing) để nạp biến môi trường vào terminal:

```
`powershell
```

```
..env.ps1
```

```
`
```

Cách 2: Sử dụng Command Prompt (CMD)

1. Tạo một file tên là `env.bat` ở thư mục gốc của dự án.

2. Viết nội dung sau vào file:

```
`cmd
```

```
set DB_URL=jdbc:postgresql://localhost:5432/parking_db
```

```
set DB_USERNAME=postgres
```

```
set DB_PASSWORD=your_password
```

```
set JWT_SECRET=DEVELOPMENT_SECRET_KEY_FOR_LOCAL_TESTING_ONLY_2026_SWP391
```

```
`
```

3. Chạy file này trên cửa sổ CMD trước khi chạy ứng dụng:

```
`cmd
```

```
env.bat
```

```
`
```

Lưu ý

⚠ **CỰC KỲ QUAN TRỌNG:** Tuyệt đối không được commit file `env.ps1` hoặc `env.bat` của bạn lên GitHub. Hai file này đã được khai báo trong `.gitignore` để Git tự động bỏ qua.

Các lệnh chạy & kiểm tra lỗi cục bộ

Phân hệ .NET Core API:

- Di chuyển vào thư mục dự án và chạy build để kiểm tra lỗi cú pháp:

```
`bash
```

```
dotnet build
```

```
`
```

Phân hệ Spring Boot Support API:

- Chạy bộ kiểm thử tự động tích hợp để chắc chắn không làm hỏng logic cũ:

```
`bash
```

```
mvn clean test
```

```
`
```

Phân hệ Frontend React/Vite:

- Biên dịch thử dự án để kiểm tra lỗi build:

```
`bash
```

```
npm run build
```

```
`
```